

정렬 비교 보고서 — 병합 · 퀵 · 트리

2026193016 채문원

수업에서 배운 병합 정렬과 퀵 정렬, 배우지 않은 트리 정렬을 같은 입력, 같은 측정 방법으로 비교한 결과입니다.

- 저장소: <https://github.com/Moonwon0701/hw1-sort-compare>
- 구현: `sorts/merge_sort.py` · `sorts/quick_sort.py` · `sorts/tree_sort.py`
- 실행: `py main.py` (측정 → `results/*.csv`) · `py plot.py` (그래프 → `results/*.png`) · `py -m unittest tests.test_sorts` (테스트 10 개)
- 환경: Windows 11, Intel Core Ultra 9 275HX, Python 3.12.10, matplotlib 3.11.2

1. 고른 정렬과 이유

구분	정렬	고른 이유
배운 정렬 1	병합 정렬	입력과 상관없이 늘 $O(n \log n)$. 비교의 기준선 역할
배운 정렬 2	퀵 정렬	평균은 가장 빠르지만 피벗을 잘못 고르면 $O(n^2)$. 피벗 규칙을 두 가지(중앙값 3 개 · 맨 앞)로 구현해 비교했습니다
배우지 않은 정렬	트리 정렬	이진 탐색 트리(BST)에 넣고 중위 순회로 꺼내는 정렬. 위키피디아 비교표에 있고, 퀵 정렬과 구조가 같다는 점이 흥미로워 골랐습니다

셋은 모두 $O(n \log n)$ 평균을 가지는 분할 정복 계열이라 비교가 공정합니다. 차이는 "어떻게 나누는가", "최악이 언제 오는가", "메모리를 얼마나 쓰는가"에서 납니다.

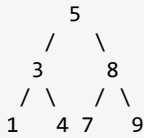
2. 트리 정렬 개념 정리

트리 정렬은 수업에서 다루지 않아 AI(Claude)에게 질문하며 공부했습니다. 공부한 개념을 항목별로 정리하고, 5 장에서 실험으로 확인했습니다.

2.1 동작 원리

- 삽입:** 원소를 하나씩 이진 탐색 트리에 넣습니다. 현재 노드보다 작으면 왼쪽, 크거나 같으면 오른쪽으로 내려가다가 빈자리에 붙입니다.
- 중위 순회:** 왼쪽 → 자신 → 오른쪽 순서로 방문하면 BST 의 성질상 오름차순이 나옵니다.

[5, 3, 8, 1, 4, 7, 9]를 넣으면 아래 트리가 되고, 중위 순회 결과는 1 3 4 5 7 8 9 입니다.

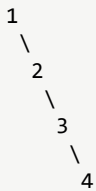


2.2 시간 복잡도

- 원소 하나를 넣는 비용은 그 원소가 놓일 깊이만큼의 비교입니다. 그래서 전체 비용은 트리 모양이 정합니다.
- 중위 순회는 노드마다 한 번씩이므로 $O(n)$ 이고, 전체 비용은 삽입이 좌우합니다.

경우	트리 모양	높이	비교 횟수
최선 · 평균 (무작위 입력)	고르게 퍼짐	$O(\log n)$	$O(n \log n)$, 평균 약 $2n \ln n \approx 1.39 n \log_2 n$
최악 (정렬됨 · 역순)	한 줄로 늘어진 사슬	n	$n(n-1)/2 = O(n^2)$

정렬된 [1, 2, 3, 4]를 넣으면 모든 원소가 오른쪽으로만 붙어 연결 리스트가 됩니다.



2.3 기본 BST와 균형 트리

- 위키피디아 비교표에서 트리 정렬의 최악이 $O(n \log n)$ 인 것은 자가 균형 이진 탐색 트리(AVL, 레드-블랙 트리)를 쓴다고 가정한 값입니다.
- 균형 트리는 삽입할 때마다 회전으로 높이를 $O(\log n)$ 으로 유지하므로 최악도 $O(n \log n)$ 이 됩니다.
- 이 과제에서는 균형을 잡지 않는 기본 BST로 구현해서 최악이 $O(n^2)$ 이고, 실험에서 그 최악을 일부러 드러냈습니다(5.2 절).

2.4 안정성

- 같은 값을 오른쪽으로 보내면 안정 정렬이 됩니다.
- 먼저 들어온 값이 트리에서 위에 있고, 나중에 들어온 같은 값은 그 오른쪽 아래에 붙습니다. 중위 순회는 오른쪽 부분 트리를 나중에 방문하므로 입력 순서가 유지됩니다.
- 구현도 `vals[i] < vals[cur]`일 때만 왼쪽으로 가고 나머지는 오른쪽으로 보냅니다. 테스트(`test_stability`)로 확인했습니다.

2.5 메모리

- 노드마다 값과 왼쪽 · 오른쪽 자식 링크가 필요합니다. 이 구현은 `vals`, `left`, `right` 세 배열을 쓰므로 추가 메모리가 $3n$ 칸이며, 셋 중 가장 많이 씁니다.
- 대신 배열에 다시 쓰는 횟수는 중위 순회에서 딱 n 번뿐입니다.

2.6 퀵 정렬과의 관계

트리 정렬은 "맨 앞 원소를 피벗으로 쓰는 퀵 정렬"과 같은 분할 구조를 가집니다.

- 트리의 루트는 첫 번째 원소입니다. 이후 모든 원소가 루트와 비교되어 왼쪽(작음) · 오른쪽(큼)으로 갈립니다. 퀵 정렬이 첫 원소를 피벗으로 분할하는 것과 같습니다.
- 왼쪽 부분 트리의 루트는 "루트보다 작은 원소 중 가장 먼저 들어온 것"입니다. 퀵 정렬의 왼쪽 구간에서 다음 피벗을 맨 앞에서 고르는 것과 같습니다.
- 다른 점은 **순서**뿐입니다. 퀵 정렬은 분할을 한 번에 끝내고 재귀하지만, 트리 정렬은 원소가 하나 들어올 때마다 그 원소의 분할 경로를 끝까지 따라갑니다. 그래서 둘의 최악도 똑같이 "정렬된 입력"입니다(5.3 절).
- 비교하는 쌍이 **정확히** 같으려면 퀵 정렬이 분할할 때 **원소의 원래 순서를 유지**해야 합니다. 그래야 각 구간의 맨 앞 원소가 "그 구간에서 가장 먼저 들어온 원소", 곧 트리의 부분 트리 루트와 일치합니다. 교환으로 순서를 섞는 분할이라면 두 번째 단계부터 피벗이 달라집니다.

2.7 쓰임새

- 배열 정렬에는 잘 쓰이지 않습니다. 퀵 · 병합 정렬보다 메모리를 많이 쓰고, 포인터를 따라 다녀서 캐시 효율도 나쁩니다.
- 데이터가 하나씩 계속 들어오면서 항상 정렬 상태를 유지해야 할 때는 유용합니다. C++의 `std::set/std::map`, Java의 `TreeMap`이 레드-블랙 트리로 이 역할을 합니다.

3. 구현

3.1 공통 규약

세 정렬은 모두 같은 모양입니다. 리스트를 제자리에서 정렬하고, 센 값을 딕셔너리로 돌려줍니다. 부르는 쪽(`bench.py`, `main.py`, 테스트)은 `SORTS` 표만 보면 되고 정렬 이름을 따로 알 필요가 없습니다.

```
SORTS = {"merge": merge_sort, "quick": quick_sort, "tree": tree_sort}
STABLE = {"merge", "tree"}
```

센 값	뜻
<code>compares</code>	원소끼리 비교한 횟수
<code>moves</code>	배열에 원소를 쓴 횟수 (퀵 정렬의 교환 1 회 = 2)

센 값	뜻
height	트리 정렬만 — 만들어진 BST 의 높이

3.2 정렬별 구현 요약

정렬	요점
병합	하향식. 보조 배열 <code>buf</code> 를 한 번만 잡아 모든 병합에서 재사용합니다. <code>buf[i] <= buf[j]</code> 로 같은 값은 왼쪽을 먼저 내보내 안정 정렬이 됩니다
퀵	피벗 규칙 <code>median3(기본) · first · random</code> . 양쪽 포인터 분할에서 피벗과 같은 값에도 멈춰 중복이 많아도 한쪽으로 쏠리지 않습니다. 작은 쪽만 재귀 하고 큰 쪽은 반복문으로 돌려 스택 깊이를 $O(\log n)$ 으로 묶었습니다
트리	노드 객체 대신 인덱스 배열 <code>left, right</code> 로 트리를 만듭니다. 정렬된 입력에서 높이가 <code>n</code> 이 되면 재귀 순회는 파이썬 재귀 한도를 넘으므로 명시적 스택 으로 중위 순회합니다

3.3 검증

`py -m unittest tests.test_sorts` — 10 개 모두 통과했습니다.

검사	내용
기본 케이스	빈 배열, 1 개, 정렬됨, 역순, 중복, 모두 같은 값, 음수, 실수
크기 훑기	<code>n = 0..300</code> 전부, 파이썬 <code>sorted()</code> 결과와 대조
제자리 · 통계	같은 리스트 객체를 정렬하는지, <code>compares/moves</code> 를 돌려주는지
안정성	병합 · 트리에서 같은 키끼리 입력 순서가 유지되는지
병합	비교 횟수 $\leq n \log_2 n$
퀵	세 피벗 규칙 모두 정렬되는지 · 맨 앞 피벗은 정렬된 입력에서 $n^2/4$ 회 이상 · 중앙값 피벗은 $20n$ 회 미만
트리	정렬된 입력에서 높이 = <code>n</code> , 비교 = $n(n-1)/2$ · 무작위 입력(<code>n = 10,000</code>)에서 높이 < 60

그 밖에 `bench.py` 의 `measure` 는 **측정할 때마다** 결과를 `sorted()`와 대조해서, 틀리면 바로 멈춥니다.

4. 실험 방법

항목	내용
시간	<code>time.perf_counter()</code> , 입력 복사를 뺀 정렬 구간만. 여러 번 재서 중앙값
비교 · 이동	구현이 직접 셴. 입력이 고정이라 몇 번을 돌려도 같은 값
입력	무작위 · 정렬됨 · 역순 · 거의 정렬($n/100$ 쌍 교환) · 중복 많음(값 8 종류). 씨앗 20260927 고정

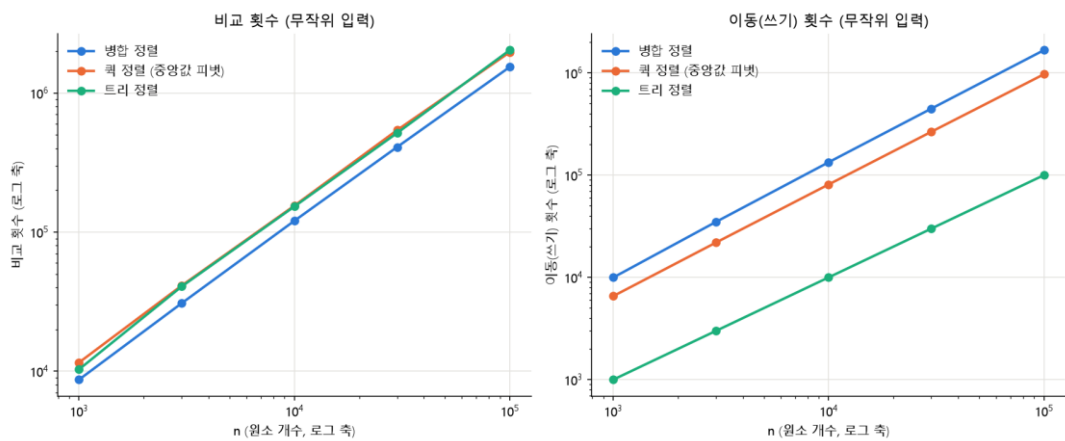
실험	대상	n	반복
① 입력 모양별	4 가지 (퀵 두 규칙 포함) × 입력 5 종	2,000	5
② n 증가	병합 · 퀵(중앙값) · 트리, 무작위	1,000 ~ 100,000	3
③ 트리 vs 맨 앞 피벗 퀵	무작위 · 정렬됨	500 ~ 4,000	1

시간은 파이썬 인터프리터 위에서 잰 값이라 구현 방식(함수 호출 수 등)의 영향이 큽니다. 알고리즘 자체를 비교할 때는 비교 · 이동 횟수를 우선해서 봅니다.

5. 실험 결과

5.1 n 이 커질 때 (무작위 입력)

n	병합 비교	퀵 비교	트리 비교	병합 ms	퀵 ms	트리 ms	트리 높이
1,000	8,711	11,561	10,323	1.17	1.07	0.76	18
3,000	30,871	41,322	40,718	4.04	3.80	2.93	26
10,000	120,552	154,912	152,799	15.34	13.92	10.94	31
30,000	408,677	540,879	515,388	50.72	47.38	37.03	35
100,000	1,536,208	1,961,861	2,031,183	191.14	171.29	156.84	41



세 정렬 모두 $n \log n$ 으로 자랍니다. 로그-로그 그래프에서 기울기를 재면 비교 횟수는 병합 1.12 · 퀵 1.12 · 트리 1.15 이고, 같은 구간에서 $n \log n$ 의 기울기는 1.11 입니다.

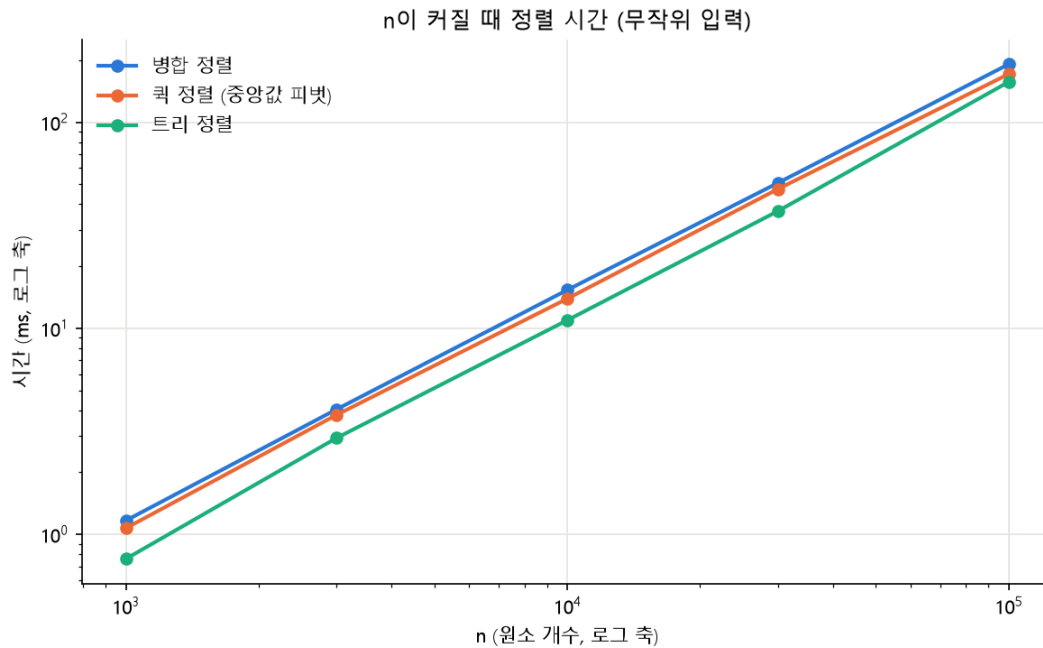
$n = 100,000$ 에서 실측값이 이론값과 거의 맞습니다.

정렬	실측 비교	이론값	이론식
병합	1,536,208	약 1,535,964	$n \log_2 n - 1.25n$
퀵 (중앙값 3 개)	1,961,861	약 1,973,000	$(12/7) n \ln n$
트리	2,031,183	약 2,018,053	$2(n+1)H_n - 4n$ (무작위 BST 평균)

비교는 병합이 가장 적고, 퀵 · 트리는 병합보다 약 30% 더 합니다. 병합은 두 구간을 언제나 정확히 반으로 나누지만, 퀵 · 트리는 피벗(루트)에 따라 분할이 한쪽으로 치우칠 수 있기 때문입니다.

이동(쓰기)은 순서가 반대입니다. 트리 100,000 회(= n , 순회에서 한 번씩) · 퀵 969,952 회 · 병합 1,668,928 회입니다. 병합은 단계(약 $\log_2 n = 17$ 단계)마다 모든 원소를 다시 쓰기 때문입니다.

트리 높이는 41 로 $\log_2 n \approx 16.6$ 의 약 2.5 배입니다. 무작위 BST 는 완전 균형은 아니어도 로그 높이를 유지합니다.

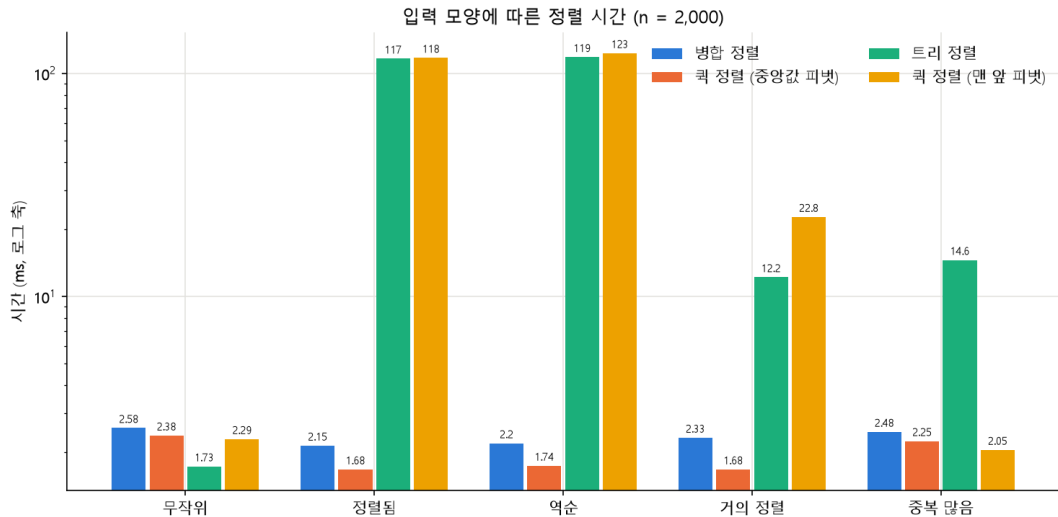


시간은 트리 정렬이 가장 빨랐습니다(156.8 ms < 퀵 171.3 ms < 병합 191.1 ms). 비교는 가장 많이 했는데도 빠른 이유는 **파이썬의 특성** 때문으로 보입니다. 퀵 정렬은 비교 · 교환마다 `less`, `swap` 함수를 호출하고, 병합 정렬은 원소마다 분기와 쓰기가 많습니다. 트리 정렬은 반복문 안에서 바로 비교하고 쓰기도 n 번뿐입니다. C 처럼 함수 호출이 가벼운 언어라면 결과가 달라질 수 있습니다. 그래서 이 보고서는 시간보다 비교 · 이동 횟수를 주요 기준으로 생각했습니다.

5.2 입력 모양별 (n = 2,000)

입력	병합	퀵 (중양값)	퀵 (맨 앞)	트리	트리 높이
무작위	19,422	25,901	27,236	23,522	22
정렬됨	10,864	22,056	2,002,998	1,999,000	2,000
역순	11,088	22,058	2,001,998	1,999,000	2,000
거의 정렬	15,516	22,056	384,886	213,397	283
중복 많음	18,629	22,675	20,796	256,217	277

(값은 비교 횟수)



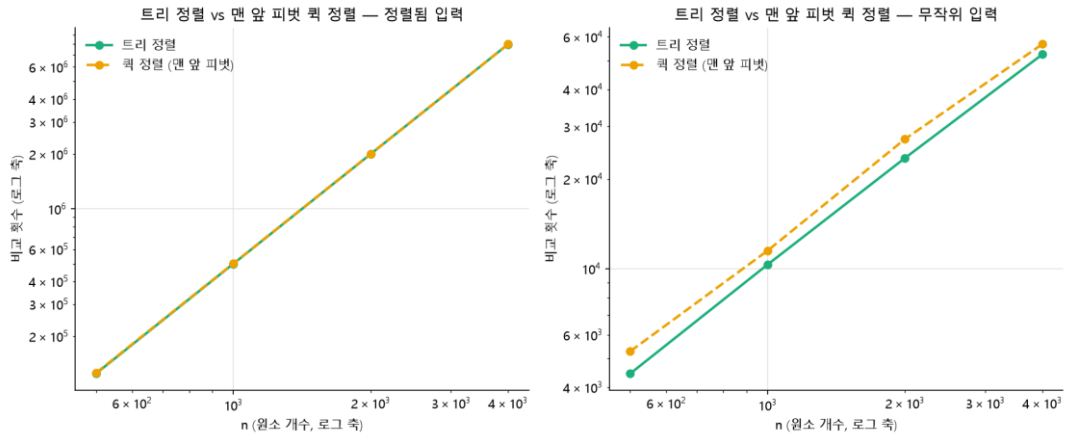
- **병합 정렬은 입력을 타지 않습니다.** 시간은 2.15~2.58 ms 로 거의 같습니다. 이동 횟수는 모든 입력에서 21,952 회로 **정확히 같습니다.** 정렬된 입력에서는 비교가 오히려 절반으로 줄어듭니다. 왼쪽 구간이 먼저 바닥나면 남은 오른쪽은 비교 없이 복사하기 때문입니다.
- **퀵 정렬(중양값 3 개)도 안정적입니다.** 정렬됨 · 역순에서는 중양값이 정확히 가운데 값이라 오히려 무작위보다 빠릅니다(1.68~1.74 ms).
- **트리 정렬과 맨 앞 피벗 퀵 정렬은 정렬됨 · 역순에서 무너집니다.** 트리는 비교 $1,999,000$ 회 = $2000 \times 1999 / 2$ 로 이론상 최악과 **정확히 일치**하고, 높이도 2,000(사슬)입니다. 시간은 약 117 ms 로 무작위(1.73 ms)의 **약 67 배**입니다.
- **거의 정렬된 입력도 위험합니다.** 20 쌍만 바꿨는데 트리 높이가 283 이고, 비교는 무작위의 9 배입니다. 현실 데이터는 "대체로 정렬된" 경우가 많아서 이 약점은 실제로 문제가 됩니다.
- **중복이 많으면 트리 정렬만 느려집니다.** 값이 8 종류뿐이라 같은 값 약 250 개가 오른쪽으로 한 줄 사슬을 이룹니다. 비교가 대략 $8 \times 250^2 / 2 = 250,000$ 회로 실측 256,217 회와 맞습니다. 안정성을 위해 같은 값을 오른쪽으로 보낸 것이 그대로 비용이 됐습니다. 퀵 정렬은 같은 값에서도 포인터가 멈추도록 분할해서 맨 앞 피벗도 이 입력에 강합니다(20,796 회).

5.3 트리 정렬 = 맨 앞 피벗 퀵 정렬?

2.6 절의 내용을 실험으로 확인했습니다.

입력	n	트리 비교	퀵(맨 앞) 비교	트리 ms	퀵(맨 앞) ms
정렬됨	500	124,750	125,748	7.0	7.2
정렬됨	1,000	499,500	501,498	28.8	29.5
정렬됨	2,000	1,999,000	2,002,998	117.6	118.1
정렬됨	4,000	7,998,000	8,005,998	477.5	472.2
무작위	500	4,444	5,279	0.34	0.47

입력	n	트리 비교	퀵(맨 앞) 비교	트리 ms	퀵(맨 앞) ms
무작위	4,000	52,422	56,734	3.75	4.95



- 정렬된 입력에서 두 선이 완전히 겹칩니다. 이 입력에서는 분할 중에 교환이 일어나지 않아 원소 순서가 그대로 유지되므로, 두 정렬이 비교하는 쌍이 같습니다. 비교 횟수 차이는 정확히 $2n - 2$ (예: $n = 4,000$ 에서 7,998 회)인데, 분할할 때 두 포인터가 엇갈리는 지점에서 퀵 정렬이 추가로 하는 비교입니다. n 이 두 배가 될 때마다 비교는 4 배로 늘어 둘 다 $O(n^2)$ 입니다.
- 무작위 입력에서는 두 선이 나란히 가지만 겹치지는 않습니다(기울기는 같습니다). 이 구현의 퀵 정렬은 양쪽 포인터로 원소를 교환하며 분할하므로 구간 안의 순서가 섞입니다. 그래서 두 번째 단계부터 구간의 맨 앞 원소(다음 피벗)가 트리의 부분 트리 루트와 달라지고, 비교하는 쌍 자체가 달라집니다. 여기에 포인터가 엇갈릴 때의 추가 비교가 더해져 트리가 7~16% 적습니다.
- 그래도 섞인 구간 역시 무작위 순서라서 피벗이 "무작위 원소"라는 성질은 같습니다. 그래서 평균 비교 횟수는 둘 다 약 $2n \ln n$ 으로 같은 속도로 자라고, 차이는 n 이 커질수록 줄어듭니다 ($n = 500$ 에서 16%, 4,000 에서 7.6%).
- 결국 두 정렬이 같은 분할 구조를 다른 순서로 실행한 것이라는 점이 정렬된 입력에서는 정확히, 무작위 입력에서는 평균적으로 확인됐습니다.

6. 결론과 한계

6.1 한눈에 비교

	병합 정렬	퀵 정렬 (중앙값 3 개)	트리 정렬 (기본 BST)
평균 시간	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
최악 시간	$O(n \log n)$	$O(n^2)$ (드물)	$O(n^2)$ (정렬된 입력 — 흔함)
비교 횟수 (무작위)	가장 적음	병합 + 약 28%	병합 + 약 32%
쓰기 횟수 (무작위)	가장 많음 ($n \log n$)	중간	가장 적음 (n)

	병합 정렬	퀵 정렬 (중앙값 3 개)	트리 정렬 (기본 BST)
추가 메모리	n (보조 배열)	$O(\log n)$ (재귀 스택)	$3n$ (값 + 링크 2 개)
안정 정렬	O	X	O (같은 값은 오른쪽)
입력 모양 민감도	없음	낮음	높음 (정렬됨 · 거의 정렬 · 중복)

6.2 결론

1. 무작위 입력에서는 셋 다 $n \log n$ 이고 이론값과 실측이 잘 맞습니다. 비교는 병합이 가장 적고, 이 파이썬 구현에서 시간은 트리가 가장 빨랐습니다.
2. 입력이 바뀌면 차이가 크게 벌어집니다. 병합 · 퀵(중앙값)은 어떤 입력에서도 2 ms 안팎인데, 기본 트리 정렬은 정렬된 입력에서 약 67 배 느려집니다.
3. 트리 정렬은 맨 앞 피벗 퀵 정렬과 같은 분할 구조를 가집니다. 순서를 유지하는 분할이라면 비교하는 쌍까지 같고, 이 구현처럼 교환하는 분할이라도 평균 비교 횟수는 똑같이 약 $2n \ln n$ 으로 자랍니다.
4. 배열을 한 번 정렬할 때는 병합이나 퀵이 낫습니다. 트리 정렬(균형 트리)은 데이터가 계속 들어오면서 정렬 상태를 유지해야 할 때 의미가 있습니다.

6.3 한계

- 균형 트리는 구현하지 않았습니다. AVL 이나 레드-블랙 트리로 바꾸면 최악이 $O(n \log n)$ 이 되는지는 확인하지 못했습니다.
- 시간은 파이썬 구현 방식의 영향이 큼니다. 함수 호출 비용 등으로 순위가 바뀔 수 있어 비교 · 이동 횟수를 함께 봤습니다.
- 한 기계에서만 했습니다. 시간은 다른 환경에서 달라질 수 있습니다(비교 · 이동 횟수는 같습니다).
- 트리 정렬의 깊이는 height 로 썼지만 메모리는 직접 재지 않았습니다. $3n$ 칸은 코드에서 계산한 값입니다.

7. 배운 점

1. 같은 $O(n \log n)$ 이라도 입력에 따라 전혀 다르게 움직입니다. 평균 복잡도만 보면 세 정렬이 같지만, 정렬된 입력 하나로 기본 트리 정렬은 67 배 느려졌습니다. 알고리즘을 고를 때는 평균뿐 아니라 최악이 언제 오는지, 그 입력이 현실에서 흔한지를 함께 봐야 한다는 것을 배웠습니다.
2. 시간만 재면 알고리즘이 아니라 구현을 비교하게 됩니다. 비교를 가장 많이 한 트리 정렬이 파이썬에서는 가장 빨랐습니다. 비교 · 이동 횟수를 직접 세어 두니 알고리즘의 차이와 구현(함수 호출 비용)의 차이를 나눠서 설명할 수 있었습니다.

3. **이론값으로 구현과 측정을 검증할 수 있습니다.** 정렬된 입력에서 트리 정렬의 비교 1,999,000 회가 $n(n-1)/2$ 와 정확히 맞았고, 무작위 입력의 비교 횟수도 세 정렬 모두 이론식과 1% 안쪽으로 맞았습니다. 숫자가 식과 맞는 것을 확인하고 나서야 결과를 믿을 수 있었습니다.
4. **새로 배운 정렬도 이미 아는 정렬과 이어집니다.** 트리 정렬이 맨 앞 피벗 퀵 정렬과 같은 분할 구조라는 것을 알고 나니, 약점(정렬된 입력)과 해결책(피벗 선택 ↔ 트리 균형)이 한 번에 이해됐습니다. 이번 과제에서 가장 큰 수확이었습니다.
5. **구현의 작은 선택이 성능과 정확성을 좌우합니다.** 안정성을 위해 같은 값을 오른쪽으로 보낸 선택 때문에 중복이 많은 입력에서 트리 정렬만 느려졌습니다. 퀵 정렬에서 피벗과 같은 값에도 포인터를 멈추게 한 것, 트리 순회를 재귀 대신 명시적 스택으로 바꾼 것도 같은 종류의 선택이었습니다.